

Megatron Core MoE 技术报告全文翻译（第四部分）

第5节 MoE的FP8/FP4低精度训练

前几节针对三堵墙分别给出了定向优化。**低精度训练**的独特之处在于：它同时影响内存、通信和计算效率，因此值得专门论述。

混合精度训练（BF16）是高效深度学习的基石。FP8/FP4低精度训练则更为激进——将精度从16位降至8位甚至4位，收益与风险都随之放大。对DeepSeek-V3而言，FP8训练每GPU节省约16 GB激活内存，通过更快的Tensor Core加速专家GEMM，并加速参数AllGather通信。这些收益叠加在三堵墙上，使FP8成为高效大规模MoE训练的关键。

然而，低精度训练会引入收敛风险，必须系统性地加以解决。本节介绍Megatron-Core的低精度训练方案：理解哪里需要精度、选择合适的量化配方，以及实现MoE特有的优化，在获取低精度收益的同时保持训练稳定性。

5.1 低精度训练对MoE的重要性

MoE架构放大了低精度训练相比稠密模型的收益与风险：

收益放大。 专家数量多，激活内存成比例增长，FP8/FP4激活带来的绝对内存节省远超稠密模型。参数AllGather的通信量减半，而主导MoE计算量的专家GEMM也受益于更快的Tensor Core吞吐。

风险放大。 路由决策依赖精确的分数来将token分配给专家。量化噪声可能破坏专家选择的稳定性，导致训练不稳定、模型质量下降或专家崩溃。数值敏感的组件必须避免激进量化。

策略：选择性精度。 解决方案是：关键处保留精度，其余处追求效率。三条原则指导MoE低精度训练：

1. **保护路由决策**——路由器保持FP32，确保专家选择稳定。
2. **关键组件保留精度**——嵌入层、输出层、主梯度、主权重和优化器状态保留原始精度，维持模型质量。
3. **批量计算量化**——构成大部分计算量的专家GEMM采用精心设计的量化方案（即“配方”）进行低精度训练。

注： MoE低精度训练成功的关键在于**选择性精度**：将数值敏感组件（路由器、嵌入层、输出层、梯度和优化器状态）保持在高精度，同时对大量计算（专家GEMM）激进量化。这一策略在避免收敛问题的同时，捕获了低精度训练的大部分收益。

5.2 低精度训练对三堵墙的影响

低精度训练对三堵墙均有收益，是一种统一性优化。表8总结了相关影响。

表8：低精度训练对三堵墙的影响总结

墙	低精度训练收益
内存	激活内存减少50% (FP8) /75% (FP4)；消除BF16权重副本；BF16优化器状态
通信	参数AllGather减少50%
计算	Tensor Core GEMM更快；量化kernel开销

5.2.1 用低精度训练突破内存墙

低精度训练以三种方式减少内存占用：

激活内存（详见4.1.3节）。激活内存节省的主要来源是保存用于反向计算的线性层输入张量。将这些张量从BF16改为FP8/FP4存储，内存占用分别减少50%/75%。例如，FP8为DeepSeek-V3每GPU节省约16 GB激活内存。

参数内存。传统低精度训练维护三份参数副本：FP32主权重、BF16模型权重和FP8/FP4计算权重。我们的原生FP8/FP4方案（详见5.3.5节）消除了BF16副本，直接从FP32转换为FP8/FP4，并以低精度进行参数AllGather。

优化器状态内存（详见4.1.6节）。Adam优化器的状态（一阶和二阶动量）可以BF16存储代替FP32，将优化器内存减少50%。这与低精度训练正交，同样适用于BF16训练。

5.2.2 用低精度训练突破通信墙

FP8/FP4的参数AllGather。使用FP8/FP4主权重配合分布式优化器时，参数AllGather通信量减少50%（每参数1字节 vs 2字节）。注意事项：

- 对于NVFP4，需要同时收集行方向和列方向的FP4权重，因此AllGather通信量减少同样是50%。
- 对于MXFP8，即使使用FP8主权重，也先将FP32主权重复制到临时BF16缓冲区后再通信。原因是MXFP8在前向和反向pass中需要不同的量化方向，同时通信行方向和列方向会导致每参数传输两字节，抵消BF16通信的优势。因此MXFP8以BF16精度通信参数。

5.2.3 用低精度训练突破计算效率墙

更快的GEMM。FP8（Ada/Hopper及更新）和FP4（Blackwell及更新）Tensor Core提供比BF16更高的吞吐量，加速前向和反向pass。

权衡：量化开销。低精度训练引入额外的量化kernel，增加了CPU开销。对于细粒度MoE，众多小操作已让CPU承压，量化开销尤为突出。管理手段包括：

- **Kernel融合**：将量化与归一化、激活函数等操作融合。
- **填充/去填充与置换kernel或路由图融合**，避免显式的填充/去填充操作。
- **分组量化**：将多个输入张量的量化融合到单个kernel中。
- **CUDA Graphs**：捕获量化kernel以消除启动开销。

5.3 低精度训练配方：Per-Tensor FP8、Blockwise FP8、MXFP8与NVFP4

Megatron-Core提供三种FP8配方，均经过大规模验证（见图30）。从per-tensor到blockwise再到MXFP8的演进，反映了硬件进步和大规模部署经验的积累。

一个低精度训练配方包含：

- **数据格式。** FP8有两种格式：E4M3和E5M2，训练中常用两种组合：
 - *E4M3*：输入、权重和梯度全部使用E4M3。
 - *混合*：输入和权重使用E4M3，梯度使用E5M2。
 - FP4仅支持E2M1格式。
- **缩放粒度。** 可选per-tensor和per-block（亦称group或tile）。per-block缩放的块大小因配方而异。
- **量化范围。** 通常量化所有线性层，保留嵌入层、输出层（LM head）和优化器的原始高精度。部分通信可使用FP8/FP4（如TP AllGather、DP AllGather、EP all-to-all）。当前配方通常保持SDPA为BF16。
- **附加机制**，如随机舍入、随机Hadamard变换（RHT）等。这些机制因配方而异，需仔细验证。

5.3.1 Per-Tensor FP8配方

Per-Tensor FP8配方在Hopper和Blackwell上均支持，通常采用混合格式（输入/权重E4M3，梯度E5M2）。它计算输入张量所有值的绝对最大值（amax），并以此将张量量化为FP8。

有两种per-tensor缩放变体：

- **延迟缩放：**使用历史窗口内的amax。这打破了amax计算与缩放之间的数据依赖，性能最优，但因估计amax而有精度损失的风险。
- **当前（实时）缩放：**即时计算amax，精度更好。

我们不推荐延迟缩放（精度较差）；当前缩放精度和收敛性更好。Per-tensor缩放是迁移到FP8训练的良好起点，简单、精度尚可。

5.3.2 Hopper上的Blockwise FP8

Blockwise FP8配方对所有张量采用E4M3格式，激活和梯度以1×128 tile量化，权重以128×128块量化。Blockwise缩放粒度更细、精度更好，已在多个大规模MoE模型生产中得到验证（包括DeepSeek-V3、Minimax-M2、Ant Ling-2.0等）。因此，**Blockwise FP8是Hopper平台上推荐的FP8配方。**

Transformer Engine为blockwise配方提供了高度优化的量化kernel和GEMM kernel（通过cuBLAS），以及层级API和autocast上下文，仅需几行代码即可启用FP8训练。

5.3.3 Blackwell上的MXFP8

在Blackwell平台上，得益于第五代Tensor Core对MXFP8格式的原生支持，我们采用更细粒度的MXFP8量化方案。激活和权重均以 1×32 粒度量化，缩放因子使用E8M0格式。理论上MXFP8应具有更高精度（更细粒度缩放），且因Tensor Core原生支持而性能更好。因此，**MXFP8是Blackwell平台上默认的FP8配方**。

尽管Blackwell支持所有布局的FP8 GEMM，但由于另一方向的量化，我们需要为激活和权重额外存储列方向量化的FP8数据。

MoE训练中MXFP8配方的持续优化方向包括：

1. 分组量化以降低量化开销。
2. 将激活和量化（用于后续GEMM）与GEMM kernel融合。
3. 确保MXFP8量化、缩放因子重排和分组GEMM的整个流水线均可CUDA Graph化。

5.3.4 Blackwell上的NVFP4

NVFP4是一种4位微缩放格式，在改善数值保真度的同时保留FP4的效率优势。

NVFP4使用E2M1格式的FP4元素。**核心设计是两级微缩放**：NVFP4应用per-tensor的FP32缩放和per-block的8位E4M3缩放。张量级FP32缩放先将张量分布映射到与块缩放兼容的范围，块级E4M3缩放再将每个块映射到FP4范围。NVFP4使用16个连续元素的块，每个块的amax被缩放到FP4最大值。

稳定的NVFP4训练还依赖以下三种实践技术：

- **随机Hadamard变换（RHT）**：用于权重梯度计算，降低异常值的影响。
- **2D缩放**：具体地，使用 16×16 权重块缩放（保留FP32张量缩放），使前向和反向pass中的权重量化更一致，减少量化失配。
- **随机舍入**：用于梯度，降低FP4转换时的舍入偏差（确定性舍入会引入偏差影响收敛）。

这些技术是论文训练配方的关键部分，对大规模收敛至关重要。

5.3.5 FP8/FP4主权重：消除冗余存储

传统低精度训练维护三层参数层次：用于优化器更新的FP32主权重、作为中间表示的BF16模型权重，以及用于前向/反向GEMM计算的FP8/FP4权重。这引入了冗余的内存开销。

原生FP8/FP4通过建立从FP32主权重直接转换为FP8/FP4计算权重的路径，完全绕过BF16中间层，同时减少内存占用并加速参数AllGather。

实现原生FP8/FP4的核心挑战在于管理FP8/FP4张量所需的量化元数据。我们提供统一接口，支持不同FP8/FP4配方（延迟缩放、当前缩放、blockwise缩放、MXFP8和NVFP4），处理TransformerEngine各版本实现的差异，确保跨版本兼容性。

在分布式优化器的每个优化器步骤中，FP32参数分片根据归约后的梯度更新，随后这些更新的FP32分片直接使用适当的缩放因子量化为FP8/FP4格式。量化过程从FP32值计算amax，跨数据并行rank更

新缩放因子以保持一致性，并将量化的FP8/FP4值写入模型参数缓冲区。这条直接路径消除了维护BF16参数的内存开销——对于参数内存可达数百GB的大型语言模型尤为显著。

原生FP8/FP4主权重的另一优势是大幅减少参数AllGather通信量（见5.2.2节）。

5.4 低精度训练的MoE特有挑战与解决方案

前面的小节介绍了适用于任何深度学习模型的低精度训练基础。MoE架构为低精度训练带来了独特挑战，需要超越标准实现的专项解决方案。

5.4.1 填充与去填充的融合：动态形状对齐

FP8和FP4 GEMM要求张量维度对齐到特定倍数：per-tensor和blockwise配方要求16的倍数，MXFP8和NVFP4要求32的倍数。这些填充要求来自每种量化配方的块缩放粒度，以及TMA（张量内存加速器）访问要求GEMM点积维度保持16字节对齐。

在前向pass中，隐藏维度K是点积维度，通常由模型设计保证对齐。但在权重梯度GEMM中，点积维度对应token维度M，M是动态变化的，往往不满足对齐要求，因此需要在token维度进行零填充。为进一步支持分组量化kernel（降低CPU开销并兼容CUDA Graph），我们将每专家的token填充增大到128。

我们提出两种优化方案：

- **路由图填充**：填充路由图而非接收的token，确保每专家token数对齐，代价是发送少量额外token，避免昂贵的per-tensor填充操作。
- **将填充融入置换操作**：避免一次额外的全局内存读写，性能最优，是可用时的默认选择。

5.4.2 分组量化与分组GEMM

分组量化。对不同专家的输入张量逐一应用量化kernel，会产生大量小kernel，增加CPU开销，GPU利用率也低。为此，我们实现了**分组量化kernel**，将多个张量的量化融合到单个kernel中，大幅降低CPU开销、提升GPU利用率，且兼容CUDA Graph。

分组GEMM。详见4.3.7节关于经优化且兼容CUDA Graph的分组GEMM。

5.4.3 NVFP4量化融合

NVFP4量化因其数值稳定性技术而格外复杂，必须激进地进行kernel融合以控制量化开销。我们的NVFP4量化kernel不仅仅是简单的“缩放+转换”：它需要吸收训练配方逻辑，包括随机Hadamard变换、2D缩放和随机舍入。

其中**RHT融合**对时延最为敏感。若作为独立kernel实现，Hadamard变换会引入一次额外的全精度（BF16）全局内存读写，显著增加带宽开销。通过将RHT与NVFP4量化融合，在单个kernel中完成Hadamard变换和FP4量化，避免了额外的BF16流量。

第二个挑战是：Blackwell NVFP4 Tensor Core GEMM面向TN布局，而Wgrad使用转置的激活和梯度。因此当RHT融合到Wgrad路径的NVFP4量化中时，kernel还必须内部完成转置，而非依赖独立的BF16转置kernel。

因此，融合量化流水线必须从同一高精度输入产生多路输出：(1)用于前向GEMM的标准FP4量化；(2)用于反向/Wgrad路径的转置+RHT+FP4量化。在训练中，我们在前向pass中启动这个融合kernel生成两份FP4副本：一份立即用于前向GEMM，一份保存用于反向。原始高精度输入随后被丢弃，从而避免存储BF16激活，同时高效地为反向路径做好准备。

NVFP4中per-tensor的FP32缩放还带来进一步的复杂性：当前预训练配方中，tensor级amax是在Hadamard变换之后计算的，这意味着需要一个专用的Hadamard+amax kernel（仅计算amax，不实现变换后的BF16输出缓冲区）。结果是Hadamard变换被计算了两次——在Hadamard+amax kernel中一次，在融合量化kernel中再次。尽管重复了变换计算，但端到端仍比写出完整变换后的BF16张量到全局内存更快，因为避免了额外的高带宽BF16读写流量。

MoE的分组NVFP4量化。在全迭代CUDA Graph要求下，MoE量化路径必须是CUDA Graph安全的：host端不能依赖CPU上的动态专家token计数，只能依赖device端的每专家token张量。这意味着输入激活不能分别量化，必须实现全NVFP4流水线的分组量化kernel，NVFP4输出的内存分配必须作为整体flat缓冲区分配，而不需要知道中间形状。

关键观察是：对于密集NVFP4激活量化kernel，大部分可以复用于分组量化，因为经路由/置换后，不同专家的token在内存中是连续打包的。主要差异是保持性能和graph安全所需的实现约束：

1. 量化线程块不应跨越多个专家的token（引入显著的控制流和索引开销），因此将每个专家的token数在token维度上零填充到线程块形状的整数倍（通常为128）。
2. NVFP4转置必须逐专家进行（独立转置每个专家的打包激活，再拼接），这不等同于将整个分组缓冲区作为一个张量转置。
3. NVFP4 GEMM要求缩放因子重排：缩放因子必须被填充到128×4对齐的形状并重排为32×16布局。这一形状约束应用于per-expert GEMM，而确定所需填充大小会隐式要求CPU可见每专家token数，这对CUDA Graph不安全。为此，我们通过构造强制每专家token为128的倍数。

上述约束（1）和（3）均要求每专家token为128的倍数，需要专用的零填充kernel。为消除该开销，我们将per-expert零填充能力融入token置换kernel，使路由后的专家激活以已对齐且零填充的布局产出，下游分组NVFP4量化/GEMM kernel无需额外预处理即可假设所需对齐。

对于NVFP4配方中的per-tensor FP32二级缩放，MoE中的per-tensor二级缩放意味着per-expert二级缩放（而非所有专家共享amax），以保持训练数值稳定性。这些amax值无法提前知晓，必须从路由token在线计算，每次迭代生成不同的per-expert amax值。为高效解决这个问题，我们利用128对齐的每专家token保证，将密集输入实现改写为CUDA Graph安全的分组Hadamard+amax kernel。

第6节 长上下文MoE训练

前几节在MoE层主导计算开销的假设下讨论三堵墙。这一假设在4K–8K token的典型训练任务中成立，专家计算和all-to-all通信是主要瓶颈。然而，随着推理模型（OpenAI o1、DeepSeek-R1等）和基于RL的训练兴起，16K、64K甚至更长的序列成为新前沿。在这些长度下，MoE训练的性能特征发生根本性转变：**注意力主导计算**，三堵墙的相对重要性随之改变。本节探讨长上下文训练如何重塑MoE模型的三堵墙，以及Megatron-Core如何应对新挑战。

6.1 注意力主导：计算重心的转移

第4节建立的框架中，MoE训练面临内存、通信和计算效率三堵墙，其优化针对的是MoE层主导计算的场景。长上下文训练从根本上改变了这一假设。关键在于MoE与注意力层的计算复杂度之差：

- **MLP组件**：计算开销随序列长度线性增长， $O(s)$ 复杂度。
- **注意力组件**：SDPA（缩放点积注意力）主导计算开销，随序列长度二次方增长， $O(s^2)$ 复杂度。

例如，在64K token时，SDPA消耗69%的FLOPs，而在短序列场景中仅占10—15%。幸运的是，SDPA在FlashAttention、cuDNN等GPU加速库中已高度优化（见表9），本身不成为性能瓶颈。因此**优化重心转移到内存和通信**：只要解决这两个挑战而不引入过多开销，训练性能即可保持。

表9：DeepSeek-V3的cuDNN SDPA性能

平台	序列长度	前向TFLOPS	反向TFLOPS
Hopper	4096	553	422
Hopper	16384	638	523
Blackwell	4096	1324	1083
Blackwell	16384	1698	1298

6.2 管理激活内存增长

随序列长度增长的激活内存是长上下文训练的首要挑战。以下一组技术协同工作，源于对大规模MoE工作负载（包括DeepSeek-V3和Qwen3）的经验，是对4.1节内存优化原则的延伸。

上下文并行（CP）与张量并行（TP）。结合CP和TP将激活内存分散到多设备，支持否则会超出GPU容量的序列长度。关键原则是保持sub-sequence长度（每个CP/TP分片的序列长度）近似不变，通常为4096或8192。随序列长度扩展CP×TP，将每设备内存维持在基线水平，使工作负载类似于短上下文训练。

优化器CPU卸载。大模型中优化器状态每GPU可消耗数十GB。CPU卸载几乎可以完全回收这部分内存，代价是传输和host端优化器开销。对于256 H100 GPU上约50% MFU的DeepSeek-V3，在序列长度至少16K时，最坏情况下开销约为2%。优化器CPU卸载通常在长上下文训练中是值得的。

选择性重计算。重计算以计算换内存，关键在于基于内存-计算权衡的模块级选择性。在长上下文场景中，SDPA主导总计算量，重计算SDPA通常过于昂贵。相比之下，重计算低成本组件（如MLP相关模块）往往能带来更好的净收益。对于64K序列长度的DeepSeek-V3，SDPA贡献高达72%的总计算；重计算它会增加约18%的计算开销，导致约16%的性能损失，却仅节省9 GB内存。相比之下，重计算非SDPA组件可节省89.8 GB内存，性能影响相当或更低。因此**建议禁用core attention重计算，优先重计算其他模块**。

实际中，CP和TP是长上下文内存效率的主要机制，优化器CPU卸载和选择性重计算提供额外缓冲，在内存受限平台上尤为重要。

6.3 上下文并行 vs. 张量并行

CP和TP是长上下文训练最有效的两种策略，实践中的问题是注意力计算主导时如何组合使用它们。

两者都能减少长上下文训练中的激活内存，但通信模式和内存效应不同（见图35）：

- **CP (P2P模式)**：在CP组内以ring方式交换KV cache，与SDPA计算重叠。
- **CP (A2A模式)**：在SDPA前将张量从序列分片转换为头分片，之后恢复序列分片。
- **TP**：额外切分线性权重，减少参数内存，但在线性层引入额外的collective通信。

实践建议：

- P2P CP是更常见的CP模式。相对TP，其主要优势是通信自然与计算重叠，而TP通信通常更难隐藏。TP减少稠密参数内存，可提高SDPA效率。
- TP通常在节点内优先（通信快，内存收益强）。跨节点时P2P CP往往更优，因为TP通信开销增大而P2P重叠仍有效。
- A2A CP介于两者之间：避免P2P CP的多步ring交换，也避免TP的线性权重切分开销。实践中A2A CP常在TP通信开销高且不希望ring交换时，与TP结合用于节点内执行。

Megatron-Core支持将两种CP后端组合为**分层CP**，实现拓扑感知的CP与TP配对。实践起点：节点内使用A2A CP与TP（提高SDPA kernel性能并减少参数内存），跨节点使用P2P CP（保持通信-计算重叠）。

6.4 变长训练的序列打包

前面的小节假设每批中序列长度固定。然而实际训练场景（尤其是强化学习和有监督微调）通常涉及变长序列，带来额外效率挑战。

传统批处理要求将所有序列填充到批内最大长度，当序列长度差异较大时浪费严重。例如，若批内序列长度为4K、8K和32K，所有序列必须填充到32K，约60%的计算和内存浪费在填充token上。

为此，Megatron-LM引入**序列打包**，在单个批次中处理变长序列而无需填充浪费。进一步地，为解决变长序列导致的DP不均衡和CP低效，引入**动态上下文并行 (Dynamic-CP)**，逐微批自适应选择有效CP大小，与序列打包方案联合优化。

6.4.1 序列打包支持

Megatron-LM支持序列打包，通过将多个变长序列拼接在单个批次中处理，无需序列间填充。这通过**THD格式**（总token数 × 头数 × 维度）实现，与传统SBHD（序列 × 批次 × 头数 × 维度）格式相对。

核心机制依赖**累积序列长度追踪**，标记每个独立序列在打包张量中的起止位置，并传播到注意力机制和TE融合kernel，确保SDPA和RoPE操作尊重序列边界，同时消除填充开销。

实现支持多种训练场景：

- **强化学习**：bin-packing算法将变长轨迹打包到固定大小的bin中，最大化GPU利用率。
- **多模态训练**：视觉-语言模型使用序列打包处理不同视觉patch数量和文本token组合导致的异构序列长度。
- **上下文并行集成**：CP启用时，系统自动切换到THD格式，并提供带填充和不带填充的累积长度，处理通信对齐同时保持计算效率。

序列打包的收益在长上下文场景中尤为显著。对于思考模型的RL训练（序列长度从数百到数万token变化），序列打包可减少40–60%的内存使用，与传统填充方案相比提升1.5–2×的训练吞吐量。

6.4.2 变长训练的动态上下文并行

THD格式的序列打包消除了填充浪费，但无法消除数据并行rank之间的计算不均衡。即使打包样本拥有相同的总token数，由于点积注意力相对子序列长度的二次方复杂度，注意力工作量仍可能差异显著。

DP不均衡导致梯度同步点时GPU空闲，还可能放大pipeline-parallel bubble。此外，当CP启用时，静态CP大小通常根据批内最长打包样本的内存需求选择，迫使较短的打包样本也使用相同的CP大小，产生不必要的CP通信开销（**CP低效**）。

****动态CP (Dynamic-CP) ****通过逐微批自适应选择有效CP大小，联合确定打包方案，同时解决DP不均衡和CP低效。调整CP大小是相对轻量的操作：仅改变token切片的分区方式和注意力算子使用的CP通信组，无需任何参数重分布或优化器状态迁移。

框架在初始化时为每个rank预构建多个CP组，候选cp_size从1到 $dp \times cp$ （限于2的幂次）。运行时调度器为每个微批选择有效cp_size及对应的预构建cp_group，实现Dynamic-CP而无需动态创建通信组的开销。

Dynamic-CP设计用于THD布局，变长序列在长度约束下打包，原始批次/序列维度折叠为token维度。直接结果是微批数量不再是从global_batch_size和micro_batch_size推导的固定量，可能在迭代间变化。Dynamic-CP在原始data_iterator外围引入轻量的data_iterator_wrapper，对每个全局批次：(i)重新调度和打包序列，创建跨DP rank的均衡工作负载；(ii)为每个微批选择有效cp_size，避免对短打包样本过度切分；(iii)返回当前迭代的有效num_microbatches。

总体而言，根据benchmark，Dynamic-CP在序列长度分布高度不均衡的真实场景（如多模态训练）中带来35–60%的端到端性能提升。

6.5 小结

长上下文MoE训练代表一种不同的优化体系，第4节的基本假设需要重新审视：

- **计算重心转移**：注意力的 $O(s^2)$ 复杂度取代MoE层成为主要计算瓶颈，在>64K token时消耗>69%的FLOPs。第4.3节的MoE优化仍有价值，但不再针对主要瓶颈。
- **内存墙加剧**：激活内存随序列长度增长，需要第3节的并行策略（CP和TP）配合第4.1节的内存技术（重计算、卸载）来维持可行的内存占用。
- **通信权衡**：CP和TP之间涉及通信开销、内存节省和SDPA计算效率的细微权衡，与第4.2节以MoE为中心的all-to-all优化有所不同。

通过结合上下文并行、张量并行、选择性重计算和优化器卸载，Megatron-Core实现了跨多种序列长度的高效训练。在256 Hopper GPU上的256K序列长度实验中：DeepSeek-V3使用TP、优化器CPU卸载和选择性重计算，达到短上下文MFU的88%；Qwen3-235B-A22B使用TP、CP和选择性重计算，达到短上下文MFU的129%（超过100%是因为SDPA kernel在长序列时效率极高）。序列打包和Dynamic-CP特性进一步将这些能力扩展到RL和SFT训练的变长工作负载。